

GEN AI Week1 Home Assignment

Activities: -

1) Find 10 different model providers and their models released in last 12 months

Details to collect:

Model Provider | Model | Release Date | Cut off Date | Cost of tokens | Context Window | Type | When to use this model

Answer: -

Model Provider	Model Name	Release Date	Cutoff Date	Cost of Tokens	Context Window	Type	When to Use This Model
OpenAI	GPT-5.6 Sol	Jul-26	Feb-26	\$2.50 / \$15.00	1.05M tokens	Proprietary / Closed	Flagship all-rounder; highly advanced agentic coding, multi-turn reasoning, and complex math/science problem-solving.
Anthropic	Claude Sonnet 5	Jun-26	Mid-2026	\$2.00 / \$10.00 (introductory)	1M tokens	Proprietary / Closed	Industry-leading choice for enterprise-level repo reasoning, code review, legal analysis, and long-context processing.
Google	Gemini 3.5 Flash	Jul-26	Jan-26	\$1.50 / \$9.00	1M tokens	Proprietary / Closed	Optimized for speed at a lower cost; ideal for high-frequency video understanding, parsing massive PDFs, and low-latency audio inputs.
DeepSeek	DeepSeek V4 Pro	Mid-2026	Jan-26	\$0.435 / \$0.87	1M tokens	Open-Weights	Best price-to-performance ratio for coding assistants, deep analytical thinking, and running complex reasoning models at scale.
xAI	Grok 4.5	Jul-26	Mid-2026	Custom API / Tiered	1M+ tokens	Proprietary / Closed	Real-time social data analysis, advanced code-generation natively integrated into IDE environments like Cursor.
Alibaba Cloud	Qwen 3.8-Max	Aug-26	Mid-2026	Competitive Open/Cloud	512K+ tokens	Open-Weights Hybrid	Exceptional performance in mathematics, logic, multi-lingual tasks, and open-source hosting for enterprise apps.
Mistral AI	Mistral Large 3	Late 2025	Late 2025	Tiered Cloud API	128K tokens	Open-Weights	Strong native multilingual tasks, localized European compliance, data privacy constraints, and self-hosted private servers.
Microsoft	Phi 5	Late 2025	Late 2025	Free / Edge Compute	32K–128K tokens	Small Language Model (SLM)	On-device, edge computing scenarios where low memory usage and local privacy are required (e.g., mobile devices offline).
Meta	Llama 4	Early 2026	Early 2026	Free (Open Source Weights)	128K+ tokens	Open Source / Weights	Building highly tailored, fine-tuned, or domain-specific production tools without relying on cloud-based API vendors.
Cohere	Command A	Early 2026	Early 2026	Enterprise Enterprise-level	128K tokens	Proprietary (Enterprise Focused)	Large-scale Retrieval-Augmented Generation (RAG) deployments, customer support automation, and corporate internal search engines.

2) Read 2–3 articles about how transformer works?

Goal to understand:

(a) Self-attention mechanism

(b) Vector embedding

(c) Feed forward Nueral Network

Answer: -

Transformer is a neural network architecture used for various machine learning tasks, especially in natural language processing and computer vision. It focuses on understanding relationships within data to process information more effectively.

- Uses attention mechanisms to capture relationships between inputs
- Processes entire sequences at once instead of step by step
- Improves performance on tasks involving context and dependencies
- Widely used across NLP, vision and other AI applications.

What are Transformers?

- Neural network architecture for NLP & vision
- Introduced in 2017: "Attention is All You Need"
- Processes whole sentences in parallel

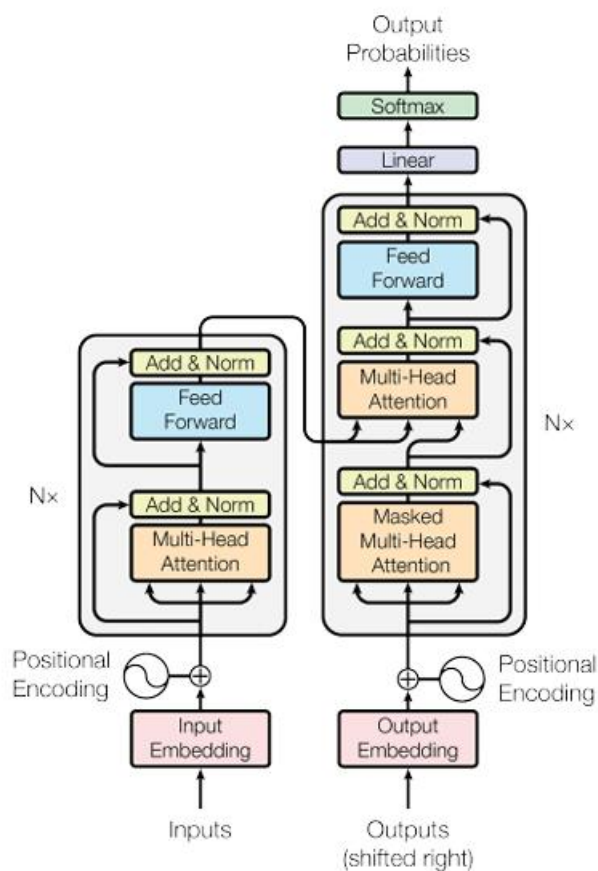
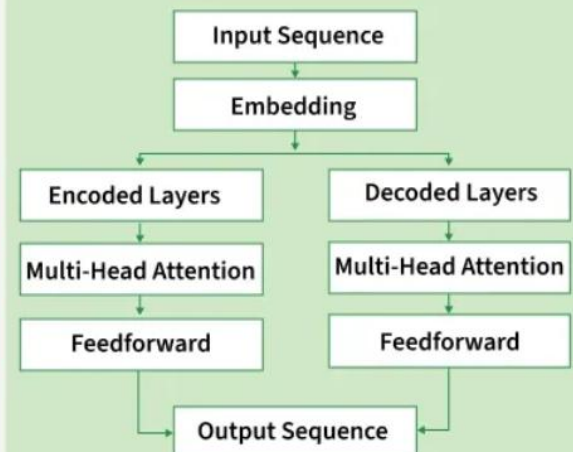


Figure 1: The Transformer - model architecture.

1. Self-Attention Mechanism: -

Self-attention helps a model understand which parts of a sentence are important when making predictions. It does this by computing three types of vectors for each word in the sentence:

- Query (Q): what this token is looking for
- Key (K): what this token offers/matches
- Value (V): information this token contributes

The model compares each token's Query with every token's Key, converts the scores into weights with softmax, then forms a weighted sum of the Values:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Following example sentence: -

Self-attention lets each word/token look at other tokens in the same sentence and decide which ones are important for understanding itself.

"The animal didn't cross the street because it was tired."

To understand "it", the model should give more attention to "animal" than to "street."

How it works:

Each token starts as an embedding vector.

The model creates three learned versions of it: Query: what this token is looking for

Key: what information this token can be matched by

Value: the information it can contribute

For one token, its Query is compared with every token's Key.

The resulting similarity scores are normalized into attention weights.

The model combines all Value vectors using those weights.

So, the vector for "it" becomes a mixture weighted heavily toward relevant words like "animal."

A Transformer uses multiple attention heads, meaning it performs this process several times in parallel. Different heads can learn different kinds of relationships—such as word reference, grammar, nearby context, or long-distance dependencies.

2. Vector embeddings: -

A neural network cannot directly operate on words; it operates on numbers. An **embedding** map every token ID to a dense vector such as:

"cat" → [0.12, -0.73, 0.44, ...]

These values are learned during training. Tokens that occur in similar contexts tend to acquire vectors that are closer in the embedding space, meaning the geometry carries useful semantic relationships.

Two useful distinctions:

- **Input/static embedding:** the initial learned lookup vector for a token.
- **Contextual embedding:** the token's vector after Transformer layers have updated it using surrounding tokens. Thus “bank” in “river bank” and “bank loan” can end up with different contextual representations.

Because attention alone does not know token order, a Transformer adds **positional information** to each input embedding. Without it, “dog bites man” and “man bites dog” would look like the same unordered set of tokens.

3. Feed-forward neural network (FFN)

After self-attention mixes information **between tokens**, the FFN processes each token's updated vector **independently** with the same small neural network.

Conceptually:

$$\text{FFN}(x) = W_2 \sigma(W_1 x + b_1) + b_2$$

It usually:

1. Projects the vector to a wider hidden space.
2. Applies a nonlinearity (such as ReLU or GELU).
3. Projects back to the model's embedding size.

A helpful mental model:

- **Self-attention:** “Who should this token consult?”
- **FFN:** “Now that it has the relevant context, how should this token transform what it knows?”

The original architecture applies the same FFN separately at every position; it does not directly mix tokens. That token-to-token mixing is attention's job.